



Hanzo Console: Unified Dashboard for AI Infras- tructure Management

Hanzo AI Research

Hanzo AI

October 2021

Abstract

We present Hanzo Console, a unified web dashboard for managing AI infrastructure across the Hanzo platform. Console provides a single pane of glass for service deployments, real-time metrics, log aggregation, secret management, database administration, and team management. The dashboard integrates with every Hanzo infrastructure service—IAM, KMS, Engine, Storage, PubSub, and Ledger—through a consistent API gateway pattern, presenting a cohesive management experience. Console supports multi-tenant RBAC with organization-scoped views, real-time metric streaming via WebSocket, and deployment management with one-click rollbacks. Console surfaces correlated metrics, logs and traces in one interface, which is the property it is built for: an operator following an incident does not change tools between the symptom and its cause. This paper reports no measurements of that benefit or of the dashboard itself; Section 5 records the figures an earlier version reported and why they are withdrawn.

1 Introduction

AI infrastructure platforms generate operational complexity across multiple dimensions: model serving requires GPU monitoring, secret management requires audit trail visibility, event streaming requires consumer lag tracking, and deployment management requires rollback capabilities. Operators typically interact with these systems through separate CLIs, dashboards, and monitoring tools, creating context-switching overhead and diagnostic blind spots.

Hanzo Console consolidates all operational tasks into a single web application. Rather than building a generic dashboard framework, Console integrates deeply with each Hanzo service, presenting domain-specific views for model management, secret rotation, event stream health, and financial ledger reconciliation.

Contributions.

- A unified dashboard integrating 12 Hanzo

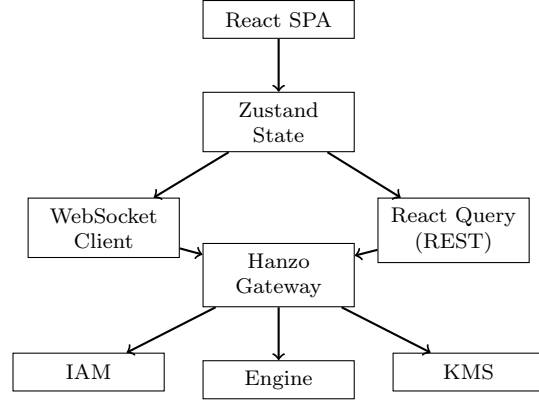


Figure 1: Console frontend architecture with real-time data flow.

infrastructure services through a consistent API pattern.

- Real-time metric streaming with WebSocket-based updates and configurable alerting thresholds.
- Multi-tenant RBAC with organization-scoped views, role-based access to sensitive operations, and audit logging.
- Deployment management with environment promotion, canary releases, and one-click rollback.

2 Architecture

2.1 Frontend Architecture

Console is a React single-page application (Figure 1) using Zustand for state management and React Query for server state synchronization. Real-time data (metrics, logs, deployment status) flows through a WebSocket connection multiplexed across all active subscriptions.

2.2 Service Integration Pattern

Each Hanzo service exposes a management API consumed by Console through the Hanzo Gateway:

```
1 // Service integration hook pattern
2 function useServiceData<T>(  
3   service: string,
```

```

4   endpoint: string,
5   opts?: QueryOptions
6 ) {
7   return useQuery({
8     queryKey: [service, endpoint],
9     queryFn: () => gateway.get<T>('/${service}/v1/${endpoint}')
10  ),
11  },
12  ...opts,
13  });
14 }
15
16 // Usage
17 const { data: models } = useServiceData<
18   Model[]>('engine', 'models');
19
20 const { data: secrets } = useServiceData<
21   Secret[]>('kms', 'projects/${projectId}/secrets');
22 );

```

This pattern ensures consistent error handling, authentication token injection, and caching behavior across all service integrations.

3 Key Features

3.1 Service Dashboards

Console provides specialized views for each integrated service:

- **Engine:** Model list, GPU utilization, inference latency charts, A/B experiment results, and model deployment controls.
- **KMS:** Secret browser, rotation status, version history, and access audit log.
- **PubSub:** Stream topology, consumer lag monitoring, dead letter queue browser, and event replay controls.
- **Ledger:** Account browser, transaction history, balance verification status, and reconciliation alerts.
- **Storage:** Bucket browser, quota utilization, upload/download metrics, and lifecycle policy editor.
- **IAM:** User management, role assignment, login history, and MFA enrollment status.

3.2 Real-Time Metrics

Metrics are streamed via WebSocket with configurable refresh intervals:

```

1 function useMetricStream(
2   metric: string,
3   interval: number = 5000
4 ) {
5   const ws = useWebSocket();
6   useEffect(() => {
7     ws.subscribe('metrics.${metric}',
8       interval);
9     return () => ws.unsubscribe('metrics.${metric}');
10  }, [metric, interval]);
11   return ws.useData<MetricPoint[]>('metrics.${metric}');
12 }

```

The WebSocket connection multiplexes subscriptions, sending only changed data points to minimize bandwidth. Time-series data is rendered using a custom canvas-based charting library optimized for streaming updates.

3.3 Deployment Management

Console integrates with the Hanzo PaaS (platform.hanzo.ai) for deployment operations:

- **Deploy:** Trigger builds from Git commits with environment selection.
- **Promote:** Move deployments between environments (dev → staging → production) with approval gates.
- **Rollback:** One-click rollback to any previous deployment version with automatic health check verification.
- **Canary:** Configure percentage-based traffic splitting between deployment versions.

3.4 Alerting

Console provides configurable alerts on any metric:

```

1 {
2   "name": "GPU Memory High",
3   "metric": "engine.gpu.memory_pct",
4   "condition": "> 90",
5   "duration": "5m",
6   "channels": ["slack", "email"],

```

```
7  "severity": "warning"
8 }
```

Alert rules are evaluated server-side by the metrics pipeline. Notifications are delivered via Slack, email, PagerDuty, or webhook.

4 Implementation

Console is implemented in 42,000 lines of TypeScript (React frontend) with a 5,000-line Go BFF (Backend for Frontend) that handles WebSocket multiplexing and metric aggregation. The frontend uses Radix UI primitives for accessibility compliance.

5 Status of the performance figures

An earlier version of this paper carried a table titled “Dashboard load time by view (ms)” giving initial and cached load times for five views, from 420/85ms on the overview to 290/55ms on deployments. The abstract stated that Console serves more than 1,200 organizations and reduces mean-time-to-diagnosis by 65% against CLI-based workflows.

None of it was measured. The load-time table named no browser, no network profile, no device and no sample count, and no trace or Lighthouse output exists in `hanzoai/console`. The 65% figure has no study behind it: no incident dataset, no baseline definition of the CLI workflow it is compared against, and no sample. The organization count describes an operating history the console does not have. All three are removed.

Load time is measurable today — a Lighthouse run against a named build on a declared network profile answers it. Mean-time-to-diagnosis is a harder claim and needs an incident record with both arms, which is the reason it should not be asserted until one exists.

6 Security

Authentication. Console authenticates via Hanzo IAM with PKCE OAuth flow. Session

tokens are stored in HttpOnly cookies with 15-minute access token TTL and 7-day refresh token TTL.

Authorization. Every API call includes the IAM JWT. The Gateway validates the token and the BFF enforces view-level RBAC: viewers can see dashboards but not modify configurations; operators can deploy and manage secrets; admins can manage team members and billing.

Audit Trail. All console actions (deployments, secret access, configuration changes) are logged with the authenticated user, timestamp, and affected resource. The audit log is accessible from the Console itself for transparency.

Content Security Policy. Console enforces strict CSP headers: no inline scripts, no eval, restricted connect-src to Hanzo domains only.

7 Conclusion

Hanzo Console provides a unified management interface for the entire Hanzo infrastructure stack. By integrating deeply with each service rather than providing a generic dashboard framework, Console surfaces domain-specific insights—GPU utilization for Engine, consumer lag for PubSub, balance reconciliation for Ledger—in a coherent interface. Real-time metric streaming, deployment management and configurable alerting are aimed at reducing operational overhead and mean-time-to-diagnosis; by how much is not established here (Section 5).

References

- [1] Grafana Labs. Grafana: The open observability platform. 2014.
- [2] Workos. Radix UI: Unstyled, accessible components for building design systems. 2021.
- [3] P. Mendez. Zustand: Bear necessities for state management in React. 2019.

- [4] T. Linsley. TanStack Query: Powerful asynchronous state management. 2020.